

INF2008 – Finals Cheat Sheet

Timothy Chia

April 07, 2026

1 CRISP-DM Framework

Definition: The CRoss-Industry Standard Process for Data Mining (CRISP-DM) is a widely used framework that provides a structured lifecycle for planning and executing data mining and analytics projects. The methodology is highly iterative, meaning that moving back and forth between its different phases is common and expected.

1. Business Understanding

This initial phase focuses on defining the project objectives, constraints, and success criteria from a business perspective rather than a technical one. It requires clearly identifying the existing problem, the decision to be improved, the evaluation metric, the ML intervention, and the quantifiable business value. In a production system, this translates into system configurations.

2. Data Understanding

This phase involves data ingestion and exploratory analysis to understand the data's characteristics, limitations, and overall quality. In production systems, this maps to data collection components.

3. Data Preparation

This step focuses on data cleaning, schema validation, and constructing feature pipelines. It translates to the data verification and feature extraction components of a production system.

4. Modelling

This phase involves the actual algorithm development, logic training, and parameterisation. It represents the core ML code within the broader system framework.

5. Evaluation

Before a model is approved for use, this phase involves offline evaluation, error analysis, and validation. In a production environment, this is handled by dedicated analysis tools.

6. Deployment

The final phase involves rolling out the model for real-world application. In production, this requires vast surrounding infrastructure, including serving infrastructure, machine resource management, workflow orchestration, and post-deployment monitoring to observe system behaviour and track performance drift.

2 Variable Types

Variable Roles

Identifier: These are unique identifiers for a row (like a Listing Number or Wafer ID) and should not be used as meaningful features in a model.

Feature (Independent): These are the inputs provided to a model that describe the situation or factors believed to influence an outcome, such as age or distance.

Target (Dependent) Variable: This is the outcome or label that the model is trying to predict or explain, such as unit price or loan approval.

Cardinal Data – Quantitative

These represent quantities where the values have magnitude, differences are meaningful, and arithmetic operations (like adding or averaging) make sense.

Discrete: Consist of countable integers, such as the number of rooms or defect counts.

Continuous: Can take any real value within a range, such as temperature, distance, or unit price.

Categorical Data – Qualitative

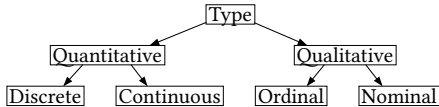
These represent labels or groupings that partition data into classes, where arithmetic operations are not meaningful.

Ordinal: Categories that have a meaningful sequence or order, though you cannot assume the mathematical spacing between them is equal (e.g. customer satisfaction ratings from "Poor" to "Excellent").

Nominal: Categories that have no intrinsic order or ranking, making them interchangeable labels like colour, location, or lot number.

Other Types: Variables can also take the form of text or booleans, and it is important to watch out for the "looks numeric" trap, where variables like postal codes appear as numbers but are actually categorical.

Diagram



3 Data Quality Issues

Missing Data

Datasets often contain missing values (NaN), empty strings, or inconsistent entries. Unhandled missing values can cause silent bias, as many tools automatically drop these rows, skewing averages and comparisons.

High Cardinality

Categorical variables with too many unique values (like lot numbers or postal codes) can cause one-hot encoding to explode the number of features.

Improper Feature Use

Using unique row identifiers (like `wafer_id` or `Listing Number`) as features is a common trap, as they do not represent meaningful predictive attributes.

"Looks Numeric" Trap

Variables like postal codes may appear as numbers but are actually categorical and should not be treated as mathematical quantities.

3.1 Imputation Strategies

Strategy A (Drop Rows): Dropping rows with missing values preserves semantic correctness but reduces the sample size and can introduce selection bias if the data is systematically missing.

Strategy B (Contextual Imputation): Filling missing values with contextual statistics (such as a mean, median, or mode) retains data rows but introduces artificial smoothing and weakens real relationships in the data.

Strategy C (Flagged Imputation): This is the preferred method. It fills missing values (like Strategy B) but also adds a binary flag column (for example, 1 if imputed and 0 if original). This preserves data, avoids silent bias, ensures transparency, and allows the model to learn if imputed rows behave differently.

3.2 Sanity Checks

Plausibility

Verify that values are within realistic bounds, ensure minimums and maximums make sense, check for impossible combinations (for example, a 1-room flat with 3 bathrooms), and make sure units are consistent.

Consistency

Check that categories are spelled consistently (for example, `AMK` vs `Ang Mo Kio`) and that group-level summaries do not contradict real-world expectations.

3.3 Feature Leakage

Leakage occurs when a model is given information that would not naturally be available at the time of prediction, allowing it to "see the answer".

This often happens when using post-outcome measurements or target-derived features. For example, using `resale_price_per_sqm` to predict total `resale_price`, or using a post-inspection defect count to predict an earlier-stage outcome.

Leakage typically inflates training performance but fails badly in production.

3.4 Class Imbalance and Representativeness

Representativeness

Check whether certain groups are under-represented, missing entirely, or whether

the dataset fails to reflect the real-world population of interest. Missing groups create blind spots.

Class Imbalance

This occurs when some categories are rare but important, such as churn or fraud. In such cases, raw accuracy can be misleading. A model can achieve very high accuracy while failing to detect the minority class at all.

3.5 Granularity and Aggregation

Granularity

Ensure the data's level of detail matches the task. For example, predicting daily price changes is different from predicting yearly price changes. If you have multiple records for the same entity across time, aggregation may be necessary.

Splitting / Structuring

If a single entry contains combined attributes (for example, "MRT, School, Market"), it should be split into distinct variables during preparation.

4 Entropy

Definition: Entropy measures the lack of order or randomness in a dataset. A pure node has entropy 0, while a perfectly balanced binary split has entropy 1.

$$H = - \sum_i p_i \log_2 p_i$$

Where p_i is the probability of samples belonging to the i th class.

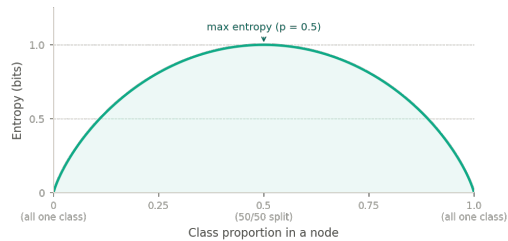
Evaluating a Split

A split is evaluated using the weighted entropy of the child nodes:

$$H_{\text{split}} = \left(\frac{n_L}{n}\right)H(L) + \left(\frac{n_R}{n}\right)H(R)$$

Lower entropy means a purer split.

Binary Entropy Curve



5 Gini Impurity

Definition: Gini impurity measures the probability that a randomly chosen data point in a node would be misclassified if you guessed its label using the class proportions of that node. A lower Gini value indicates a purer node, with 0 meaning all samples belong to a single class.

$$\text{Gini} = 1 - \sum_i p_i^2$$

Where p_i is the proportion of the i th class in the node.

Gini is often preferred in practice because it is faster to compute than entropy, since it avoids logarithms.

Evaluating a Split

To evaluate whether a split is good, compare the impurity of the parent node with the weighted impurity of the child nodes:

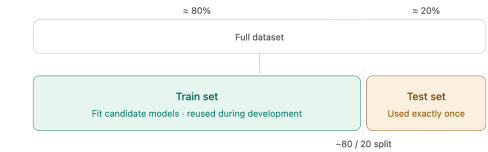
$$G_{\text{split}} = \left(\frac{n_L}{n}\right)G(L) + \left(\frac{n_R}{n}\right)G(R)$$

The best split is the one that most reduces impurity.

6 Train-Test Split

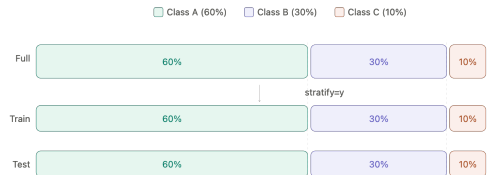
Train Set: Used to fit candidate models. It is reused during development, but training accuracy should not be used to choose hyperparameters.

Test Set: The final judge. It should be used exactly once at the very end and must not influence training or model selection.



Stratification

Stratification preserves the original class ratio in each split. This is especially important for classification tasks so that minority classes do not disappear from a split.



Rule of thumb: If the target is categorical, use `stratify = y`.

Critical Rule

Fit all transformers (imputers, encoders, scalers) on the training set only, then apply `.transform()` to validation and test data. Never let validation or test data influence preprocessing fit parameters.

```
[Incorrect – leaking fit to test data]
# Scaler fitted on the whole dataset before splitting
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
```

```
[Correct – fit on train, transform test separately]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train) # fit on train only
X_test = scaler.transform(X_test)      # apply to test
```

7 Bias-Variance Trade-off

Error Decomposition

Expected prediction error can be viewed as:

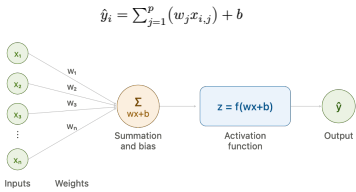
$$\text{Error} = \text{Bias}^2 + \text{Variance} + \epsilon$$

Bias is error caused by overly strong assumptions. High-bias models are too simple, miss real patterns, and underfit. Both training and test error tend to be high.

Variance is error caused by a model being too sensitive to the training data. High-variance models fit noise rather than signal, giving low training error but high test error.

With multiple linear regression as the basis for the perceptron, we can now apply it to different problem contexts. One use case of the perceptron is **Simple Linear Regression**.

Recall that in **Multiple Linear Regression**, $\hat{y}_i$ is given by:



In the perceptron diagram above, each input feature is multiplied by its corresponding weight, and these weighted inputs are then added together with the bias term. This is the part labelled **summation and bias**, which represents:

$$\sum_{j=1}^p (w_j x_{i,j}) + b$$

However, for **Simple Linear Regression**, there is only one input feature. Hence, the general perceptron expression simplifies to:

$$\hat{y}_i = wx_i + b$$

So, in this case:

- the multiple inputs in the diagram reduce to just one input x_i
- the corresponding weight reduces to a single weight w
- the summation block becomes $wx_i + b$

For simple linear regression, the activation function is just the identity function, so it does not change the value from the summation block. Therefore, the final output is simply:

$$\hat{y}_i = wx_i + b$$

The goal is to find the values of w and b that **minimise** the prediction error. This is given by:

$$\min_{w,b} \frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

13.1 Forward Pass

For each sample i , the model predicts:

$$\hat{y}_i = wx_i + b$$

The residual for sample i is:

$$e_i = y_i - \hat{y}_i$$

If $e_i > 0$, the model predicted too low.
If $e_i < 0$, the model predicted too high.

13.2 Loss Computation

We do not sum residuals directly, because positive and negative errors can cancel out. Instead, we square the errors and use the HMSE:

$$L(w, b) = \frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Since $\hat{y}_i = wx_i + b$, the loss can also be written as:

$$L(w, b) = \frac{1}{2n} \sum_{i=1}^n (y_i - (wx_i + b))^2$$

The factor $\frac{1}{2}$ is included so differentiation during backpropagation becomes cleaner. Remember, the goal is $\min_{w,b} L(w, b)$.

13.3 Backpropagation

Backpropagation computes how the loss changes with respect to the parameters w and b . Before taking partial derivatives, it helps to see the computation flow:

$$w, x_i, b \rightarrow z_i \rightarrow \hat{y}_i \rightarrow L(w, b)$$

For simple linear regression: $z_i = wx_i + b$ and $\hat{y}_i = z_i$.

So the full flow is:

$$w, x_i, b \rightarrow (wx_i + b) \rightarrow \hat{y}_i \rightarrow \frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

This shows that w and b affect the prediction first, and the prediction then affects the loss. Using the chain rule, we trace that dependency backwards:

Loss wrt b

$$\frac{\partial(L)}{\partial(b)} = \sum_{i=1}^n \frac{\partial(L)}{\partial(z_i)} \cdot \frac{\partial(z_i)}{\partial(b)}$$

Loss wrt w

$$\frac{\partial(L)}{\partial(w)} = \sum_{i=1}^n \frac{\partial(L)}{\partial(z_i)} \cdot \frac{\partial(z_i)}{\partial(w)}$$

Since $\hat{y}_i = z_i$, the residual is $e_i = y_i - \hat{y}_i$. Then the gradients become:

$$\begin{aligned} \frac{\partial(L)}{\partial(b)} &= -\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i) \\ &= -\frac{1}{n} \sum_{i=1}^n e_i \end{aligned} \qquad \begin{aligned} \frac{\partial(L)}{\partial(w)} &= -\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i) \cdot x_i \\ &= -\frac{1}{n} \sum_{i=1}^n e_i x_i \end{aligned}$$

13.4 Gradient Descent

Gradient descent updates the parameters in the direction that reduces the loss.

- Choose a learning rate $\alpha > 0$
- Update learnable parameters (w, b)

$$w := w - \alpha \cdot \frac{\partial(L)}{\partial(w)} \qquad b := b - \alpha \cdot \frac{\partial(L)}{\partial(b)}$$

- Repeat the forward pass, loss computation, and updating of parameters until convergence.

14 Activation Functions

An activation function $g(z)$ is applied to the linear computation

$$z = w^T x + b$$

at each layer of a neural network. The choice of activation function determines the network's ability to learn complex, non-linear mappings.

Motivation

If every layer uses a linear activation function

$$g(z) = z,$$

then the composition of layers collapses into a single linear transformation. For a two-layer network:

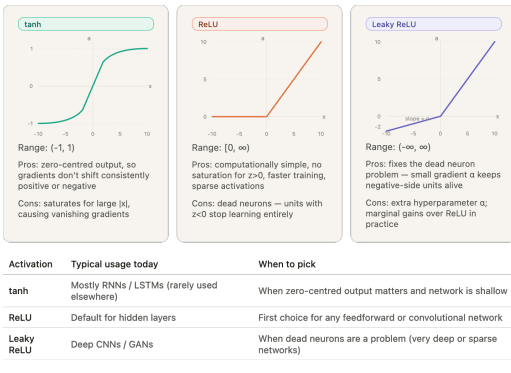
$$\begin{aligned} a^{[2]} &= W^{[2]} (W^{[1]} x + b^{[1]}) + b^{[2]} \\ &= W' x + b'. \end{aligned}$$

Stacking linear layers is equivalent to a single linear layer, so depth adds no expressive power. Non-linear activations prevent this collapse, enabling the network to learn complex functions.

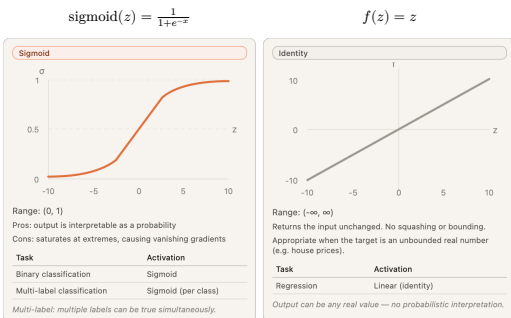
Crucially, they also allow hidden units to capture *feature interactions* – combinations of inputs that no single linear term can represent – which is what lets deeper layers build progressively richer representations.

14.1 Inner Activations

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \qquad \text{relu}(z) = \max(0, z) \qquad \text{lrelu}(z) = \max(\alpha z, z)$$



14.2 Outer Activations



Softmax

For a vector $z = [z_1, z_2, \dots, z_K]$:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Softmax is a generalisation of sigmoid to multiple classes. It takes K raw scores (logits) which can be positive, negative, or zero – and transforms them into K values between 0 and 1 that sum to 1, **forming a valid probability distribution**.

Large inputs become large probabilities; small or negative inputs become small probabilities. It can only be used when classes are mutually exclusive.

15 Distance Metrics

Euclidean Distance

Euclidean distance is given by:

$$d(x, y) = \sqrt{\sum_{i=1}^p (x_i - y_i)^2} = \sqrt{\sum (\Delta x)^2 + (\Delta y)^2}$$

Where $\Delta x = x_2 - x_1$ and $\Delta y = y_2 - y_1$ on a two-dimensional plane.

Manhattan Distance

Mahattan distance is given by:

$$d(x, y) = \sum_{i=1}^p |(x_i - y_i)| = |\Delta x| + |\Delta y|$$

Chebyshev Distance

Chebyshev distance is given by:

$$d(x, y) = \max_{1 \leq i \leq p} |(x_i - y_i)| = \max(|\Delta x|, |\Delta y|)$$

16 Normalisation

Normalisation is important to avoid the “scaling trap” caused by features of differing scales.

Without normalisation, the feature with the larger numeric range dominates distance-based metrics, which can lead to incorrect neighbours and wrong classification.

To normalise a feature x to the range of $[0, 1]$, we use the following equation:

$$z_i = \frac{x_i - \min(x)}{\max(x) - \min(x)}$$

17 Cluster Analysis Metrics

Metrics used in cluster analysis can help to tell us how well-grouped a clusters are and how distinct clusters are from one another.

Inertia Score

Inertia or **Within Cluster Sum of Squares (WCSS)** measures the sum of squared distances between each point and its cluster centroid, summed across all clusters. It is given by:

$$J = \sum_{j=1}^K \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

Where:

- J is loss function to minimise
- K is the number of clusters
- j is the cluster index from 1 to k
- x_i is the i^{th} data point in the dataset
- $x_i \in C_j$ is the i^{th} data point that belongs to cluster C_j
- μ_j is the centroid (mean vector) or cluster C_j

The mean vector μ_j is given by:

$$\mu_j = \frac{1}{|C_j|} \cdot \sum_{x_i \in C_j} x_i$$

It is considered a **global** metric, because it tells us about the **overall compactness** of all clusters, but not how **separable** the clusters are.

Silhouette Score

The **Silhouette** score is composed of two metrics:

- Cohesion**, denoted $a(i)$, measures the mean distance to **all other points in the same cluster**.
- Separation**, denoted $b(i)$, measures the mean distance to **all points in the nearest neighbouring cluster**.

Combining cohesion and separation gives us the silhouette score which is given by:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

The **average silhouette score** is given by:

$$\bar{s} = \frac{1}{n} \cdot \sum_{i=1}^n s(i)$$

Where $\forall i \in \{1, \dots, n\}$, s_i is the silhouette score or point x_i .

Silhouette Score Interpretation

$s(i) \approx +1$ Well clustered	$s(i) \approx 0$ On boundary	$s(i) < 0$ Likely misplaced
-------------------------------------	---------------------------------	--------------------------------

Distance as a Loss Function

Within each cluster C_k , we want to find a centroid μ_k such that the variance between points within C_k and μ_k is minimised. This is given by:

$$\min_{\mu_k} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

18 Clustering

A subfield of machine learning is **Unsupervised Learning**. Unlike in Supervised Learning, we instead deal with **unlabeled data** and attempt to learn partitions or groups from the unlabeled datasets.

Example

1. We are given only dataset of independent variables $X = \{x_1, x_2, x_3, \dots, x_n\}$.

2. There are no target variables or labels.

3. The goal is to learn hidden structures and group similar data points.

18.1 K-Means

Partition-based clustering method that groups points by assigning them to the nearest centroid and updating centroids iteratively.

Steps

1. Choose the number of clusters k .

2. Randomly assign k initial centroids.

3. Measure the squared distance between a point to each cluster's centroid.

4. Assign the data point to its nearest centroid's cluster.

5. Repeat steps 3-4 for all data points.

6. Recompute each cluster's centroid to find μ_k^* .

7. Repeat steps 3-6 until convergence

Convergence

1. Stop when the maximum number of iterations is reached.

2. Stop when the change in loss falls below a threshold.

3. Stop when centroid movement falls below a threshold.

Limitations

• **Must choose the number of clusters k beforehand**

• **Choosing k is fundamentally heuristic**

• **The elbow method is subjective and may not show a clear elbow**

• **Loss always decreases as k increases, so larger k can look artificially better**

• **Sensitive to the initial choice of centroids**

• **Sensitive to outliers**

• **Works best when clusters are roughly compact, spherical, and similarly sized**

• **Can struggle in high dimensions because cluster structure becomes less distinct**

• **In high dimensions, elbow curves often become smoother and harder to interpret**

• **Dimensionality reduction such as PCA is often applied before clustering**

18.2 K-Means++

Improved initialisation method for K-Means that chooses starting centroids more carefully to spread them out before clustering begins.

How it works

1. Choose the first centroid randomly.

2. For each remaining point, compute its distance $D(x_i)$ to the nearest already chosen centroid.

3. Choose the next centroid with probability proportional to the squared distance:

$$P(x_i) \propto D(x_i)^2$$

4. Equivalently,

$$P(x_i) = \frac{D(x_i)^2}{\sum_j (D(x_j)^2)}$$

5. Repeat until k initial centroids have been selected.

6. Run standard K-Means using these initial centroids.

Intuition

1. Points far from existing centroids are more likely to be selected.

2. This spreads the initial centroids across the dataset.

3. It reduces the chance that multiple centroids start in the same dense region.

Benefits

1. Usually gives better starting centroids than fully random initialisation.

2. Often converges faster than standard K-Means.

3. Can improve clustering quality by avoiding poor initial placements.

4. Helps separate well-spaced groups early in the algorithm.

Limitations

Still requires choosing the number of clusters k beforehand

Still inherits the main weaknesses of K-Means after initialisation

Can still perform poorly when clusters are non-spherical, overlapping, or have very different sizes

Sensitive to outliers, since faraway points can get high selection probability

Improves centroid initialisation, but does not guarantee the globally optimal clustering

Use Cases

1. When standard K-Means gives unstable results due to random initialisation.

2. When you want a better default initialisation for centroid-based clustering.

3. When clusters are expected to be reasonably compact and separated.

4. Commonly used as the standard practical version of K-Means.

18.3 DBSCAN

Density-based clustering method that finds dense regions and marks sparse points as outliers.

1. **Core Point**: a point with at least `min_samples` neighbours within radius `epsilon`

2. **Border Point**: a point within `epsilon` of a core point, but without enough neighbours to be a core point itself

3. **Noise Point**: a point that does not belong to any dense region and is marked as an outlier

Hyperparameters

1. `epsilon`: the radius within which points are considered neighbours

2. `min_samples`: the minimum number of points required to form a dense region (cluster)

Limitations

• Choosing the right `epsilon` and `min_samples` can be tricky

• Struggles when cluster densities differ significantly

• Large `epsilon` can over-merge clusters

• Too small `epsilon` can cause fragmentation

• Higher `min_samples` makes DBSCAN more conservative and marks more points as noise

• Lower `min_samples` can create too many small clusters

18.4 K-Means vs. DBSCAN

Feature	K-Means	DBSCAN
Requires k ?	Yes	No
Shape Assumption	Spherical / Convex	Arbitrary
Handles Noise	No	Yes
Sensitive to Scaling	Yes	Yes

Feature	K-Means	DBSCAN
Sensitive to Initialization	Yes	No
Struggles With	Irregular shapes	Varying densities
Optimization based	Yes (minimizes J)	No
Produces convex clusters	Yes	No
Works Well When	Compact clusters	Density-separated clusters